FACULTY:	**FACULTY OF ICT**
PROGRAMME:	**PROGRAM**
YEAR:	**2**
LECTURER:	**Ms. MARRY KADWA**
FROM:	**BRIGHT KISO [2420007]**
COURSE:	**OOP**
COURSE CODE:	**BSCICT 2203**
DUE DATE:	**2nd APRIL 2026**

Codes for slide set 3

Code 1:

```c
#include <stdio.h>

int main(void) {
    int arr[3] = {7, 14, 21};

    printf("%d\n\n", arr[2]);
    printf("%d\n\n", *(arr + 2));

    return 0;
} // end of main

```

Output of code:

```
C:\Users\Kamwai\Desktop\ED\Year 2\Distributed systems\Assignments\Assignment 1\Assignment 2.exe
21

21


--------------------------------
Process exited after 0.3776 seconds with return value 0
Press any key to continue . . .
```

Code function:

This program demonstrates the relationship between arrays and pointers in C.

An integer array (arr) is created and initialized with the values 7, 14, and 17. A pointer (ptr) is then declared and assigned to point to the first element of the array (arr). In C, the name of an array acts like a pointer to its first element, so (ptr) and arr refer to the same memory location.

The program then prints the second element of the array in two ways. First, it uses pointer indexing with (ptr)[1], and second, it uses normal array indexing with (arr)[1]. Both expressions access the same value stored at index 1, which is 14.

This shows that pointer notation and array indexing are interchangeable in C when accessing elements of an array.

Code 2:

```c
#include <stdio.h>

int main(void) {
    char tmp[5] = {1, 2, 3, 4, 5};
    char *arr = tmp + 4;

    printf("%d\n\n", arr[-2]);

    return 0;
} // end of main
```

Output:

```
C:\Users\Kamwai\Desktop\ED\Year 2\OOP\Codes in slide2\12\2.11.exe
3


--------------------------------
Process exited after 0.2704 seconds with return value 0
Press any key to continue . . .
```

Code Function:

This program demonstrates how pointers can be used to access array elements using both forward and backward indexing in C.

An array tmp is created and initialized with the values 1, 2, 3, 4, and 5. A pointer arr is then assigned to point to the last element of the array by using tmp + 4. Since array indexing starts at 0, tmp[4] refers to the value 5.

The program then uses pointer arithmetic in reverse by accessing arr[-2]. This means it moves two positions backward from the current pointer position. Since arr is pointing at tmp[4], moving back two steps lands on tmp[2], which contains the value 3.

Finally, the program prints this value. This shows that pointers in C can access memory in both forward and backward directions relative to a reference point, as long as the memory accessed is within the bounds of the array.

Code 3:

```c
#include <stdio.h>

void addNumber(int *ptr) { // add 100 to all values of array
    int i = 0;
    for (i = 0; i < 10; i++) {
        ptr[i] += 100;
    }
}

int main(void) {
    int i = 0;
    int arr[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

    printf(">>Before function call<<\n");
    for (i = 0; i < 10; i++) {
        printf("%4d", arr[i]);
    }

    printf("\n\n");

    addNumber(arr);

    printf(">>After function call<<\n");

    for (i = 0; i < 10; i++) {
        printf("%4d", arr[i]);
    }

    printf("\n");

    return 0;
}
```

Output:

```
C:\Users\Kamwai\Desktop\ED\Year 2\OOP\Codes in slide2\12\2.11.exe
>>Before function call<<
   1   2   3   4   5   6   7   8   9  10

>>After function call<<
 101 102 103 104 105 106 107 108 109 110

--------------------------------
Process exited after 0.1915 seconds with return value 0
Press any key to continue . . .
```

Code Function:

This program shows how a function can modify an entire array using pointers.

The array arr is initialized with values from 1 to 10. Before calling the function, the program prints all elements. The function addNumber() receives a pointer to the array and loops through all elements, adding 100 to each one directly in memory.

After the function call, the modified array is printed again. The output shows that all values have increased by 100, proving that arrays are passed to functions by reference (using pointers), meaning changes inside the function affect the original array.

Code 4:

```c
#include <stdio.h>

int main(void) {
    char c = 'a';

    printf("c: %c\n", c);
    printf("&c: %p\n\n", (void*)&c);

    return 0;
}
```

Output:

```
C:\Users\Kamwai\Desktop\ED\Year 2\OOP\Codes in slide2\12\2.11.exe
c: a
&c: 000000000062FE1F


--------------------------------
Process exited after 0.1305 seconds with return value 0
Press any key to continue . . .
```

Code Function:

This program demonstrates how to display both the value of a variable and its memory address in C.

A character variable c is declared and initialized with the value 'a'. The first printf statement uses the %c format specifier to print the actual character stored in c, which is a.

The second printf statement prints the memory address of c. The & operator is used to obtain the address of the variable, and %p is the correct format specifier for printing pointer (address) values. The address is cast to (void*) to match the expected type for %p.

Finally, the program returns 0 to indicate successful execution.

<u>Codes in slide set 4</u>

Code:

```c
#include <stdio.h>

void funcOne(void) {
    printf("funcOne was called!\n");
} // end of funcOne

void funcTwo(void) {
    printf("funcTwo was called!\n");
} // end of funcTwo

int main(void) {

    funcOne();
    funcTwo();

    printf("Address of funcOne: %p\n", (void*)funcOne);
    printf("Address of funcTwo: %p\n", (void*)funcTwo);

    return 0;
}
```

Output:

```
C:\Users\Kamwai\Desktop\ED\Year 2\OOP\Codes in slide2\4\Assignment 2.exe
funcOne was called!
funcTwo was called!
Address of funcOne: 0000000000401530
Address of funcTwo: 000000000040154B

--------------------------------
Process exited after 0.3182 seconds with return value 0
Press any key to continue . . .
```

Code function:

This program demonstrates functions and function addresses in C.

Two functions, funcOne and funcTwo, are defined. Each function prints a message when it is called. In the main() function, both functions are executed normally, showing that they work as independent blocks of code.

After calling the functions, the program prints their memory addresses using %p. In C, function names represent the starting address of the function in memory. This means funcOne and funcTwo are actually pointers to code locations.

Code 2:

```c
#include <stdio.h>

int adder(int n1, int n2) {
    printf("Adder was called: ");
    return n1 + n2;
} // end of adder

int divider(int n1, int n2) {
    printf("Divider was called: ");
    return n1 / n2;
} // end of divider

void printResult(int n1, int n2, int (*fPtr)(int, int)) {
    printf("%d\n", fPtr(n1, n2));
} // end of printResult

int main(void) {

    int num1 = 10;
    int num2 = 5;

    printResult(num1, num2, adder);
    printResult(num1, num2, divider);

    return 0;
} // end of main
```

Output:

```
C:\Users\Kamwai\Desktop\ED\Year 2\OOP\Codes in slide2\4\Assignment 2.exe
Adder was called: 15
Divider was called: 2

--------------------------------
Process exited after 0.1538 seconds with return value 0
Press any key to continue . . .
```

Code Function:

This program demonstrates the use of function pointers as parameters in C, allowing different functions to be called dynamically.

Two functions are defined: adder() and divider(). The adder() function takes two integers, prints a message indicating it was called, and returns their sum. The divider() function does the same but returns the result of dividing the first number by the second.

A third function, printResult(), is created to take two integers and a function pointer as parameters. The function pointer (int (*fPtr)(int, int)) allows printResult() to receive either adder or divider as an argument. Inside printResult(), the function pointer is used to execute whichever function was passed in, and the result is printed.

In the main() function, two integer values (num1 and num2) are declared. The printResult() function is then called twice: first with adder, and then with divider. This causes the program to execute different functions using the same interface.

Overall, the program shows how function pointers enable flexible and reusable code, where functions can be passed as arguments and selected at runtime instead of being hardcoded.

Code 3:

```c
#include <stdio.h>

void func1(void) {
    printf("func1 was called!\n");
} // end of func1

void func2(void) {
    printf("func2 was called!\n");
} // end of func2

void (*whatFunction(int sel))(void) {
    if (sel == 1)
        return func1;
    else
        return func2;
} // end of whatFunction

int main(void) {

    void (*fPtr)(void);

    fPtr = whatFunction(1);
    fPtr();

    fPtr = whatFunction(2);
    fPtr();

    return 0;
} // end of main
```

Output:

```
C:\Users\Kamwai\Desktop\ED\Year 2\OOP\Codes in slide2\12\2.11.exe
func1 was called!
func2 was called!

--------------------------------
Process exited after 0.1117 seconds with return value 0
Press any key to continue . . .
```

Code Function:

This program demonstrates returning function pointers from a function in C.

Two functions, func1() and func2(), are defined. Each function simply prints a message when called.

The function whatFunction(int sel) returns a pointer to a function based on the value of sel. If sel is 1, it returns the address of func1; otherwise, it returns the address of func2.

In main(), a function pointer fPtr is declared. It is assigned the function returned by whatFunction(1), and then called using fPtr(), which executes func1. The pointer is then reassigned using whatFunction(2) and called again, executing func2.

Code 4:

```c
#include<stdio.h>
int main(void){
    char c='a';
    char *ptr=NULL;

    ptr = &c;

    printf("ptr: %p\n", (void*)ptr);
    printf("ptr: %c\n\n", *ptr);

    return 0;
}
```

Output:

```
C:\Users\Kamwai\Desktop\ED\Year 2\OOP\Codes in slide2\12\2.11.exe
ptr: 000000000062FE17
ptr: a


--------------------------------
Process exited after 0.1197 seconds with return value 0
Press any key to continue . . .
```

Code Function:

This program demonstrates the use of pointers in C. A character variable c is declared and assigned the value 'a'. A pointer ptr is then declared and initialized to NULL, meaning it does not point to any valid memory initially.

The pointer is then assigned the address of c using ptr = &c, so it now stores the location of the variable c. The program prints the value stored in the pointer (the memory address of c) and then uses dereferencing (*ptr) to access and print the actual value stored at that address, which is 'a'.

This shows how pointers can store memory addresses and be used to access and manipulate values stored in memory directly.